HAL 9000 Technical Briefing

Prepared for the ARC technical team ahead of the May 4, 2026 firm-wide demo.

Executive Summary

HAL 9000 is being rebuilt from a local PDF assistant into a firm-wide research operating system. The goal is not another chat interface. The goal is a shared, auditable research platform where acquisition, ingestion, semantic memory, agent execution, review, collaboration, and export all operate against one canonical store.

The current rebuild branch has crossed an important threshold: HAL now has a working shared data model, repeatable research program contracts, migration support, object storage abstraction, vector retrieval, bounded worker execution, source-backed output generation, review workflows, collaboration surfaces, and a passing CI quality gate. It is ready to demo as an early but coherent research OS loop.

The demo story is intentionally simple:

1. Seed a shared project and research memory.
2. Search that memory across chunks, claims, and outputs.
3. Stage source-backed outputs from a run.
4. Review and annotate those outputs in the browser.
5. Act from Slack.
6. Sync operational views to Sheets.
7. Show the audit trail proving what happened.

Why Rebuild HAL

The original HAL workflow was useful but local-first: PDFs, extracted notes, and outputs were tied to a developer/operator workflow. That does not scale to a firm-wide research system where multiple people need to contribute, review, and reuse the same research memory.

The rebuild is designed around five engineering principles:

- Shared state beats local state. Research artifacts should live in one

canonical data store, not scattered across laptops and one-off folders.

- Programs should be contracts. A research task should declare objective, scope,

budget, tools, and required outputs before the worker runs.

- Memory should be queryable and reviewable. HAL should retrieve from previous

chunks, claims, and outputs, then show the evidence behind generated results.

- Outputs should be staged before promotion. A human reviewer should be able to

promote, reject, request changes, comment, and inspect audit history.

- Non-CLI access matters. Technical users can keep the CLI, but Slack, browser

views, and Sheets make the system usable by the rest of the firm.

Current Architecture

At a high level, the rebuilt HAL stack has six layers:

1. Research program contract: validated `program.md` files define the agent

objective, scope, budget, tool policy, and output requirements.

1. Canonical research store: SQLAlchemy models and `ResearchStore` persist

projects, programs, runs, documents, chunks, claims, evidence, outputs, review decisions, comments, notifications, saved views, graph edges, and audit records.

1. Artifact storage: a common object-store protocol supports local filesystem

development and S3-compatible production storage.

1. Vector and graph memory: deterministic fake embeddings support tests, while

the production path is Postgres plus pgvector. Retrieval now spans document chunks, extracted claims, and staged outputs. Graph edges track relationships such as cites, supports, contradicts, uses_method, studies_material, and reports_property.

1. Worker orchestration: bounded workers execute queued research runs, enforce

runtime/tool/LLM budgets, perform acquisition and corpus preparation, attach retrieval context, generate source-rich outputs, and record telemetry.

1. Collaboration surfaces: CLI, browser review UI, Slack command/action

handlers, Google Sheets sync jobs, notification delivery workers, and audit dashboards all operate over the same store and permission model.

Data Model And Provenance

The canonical store now covers the core objects needed for a firm-wide research OS:

- `ResearchProject`: shared workspace and permission boundary.
- `ResearchProgramRecord`: persisted research contract.
- `ResearchRun`: executable instance of a program.
- `Document`: source document metadata and corpus hardening fields.

- `DocumentChunk`: canonical text chunks for retrieval.
- `ExtractedClaim`: normalized claims derived from source material.
- `EvidenceLink`: lineage from claims and outputs back to chunks/documents.
- `ResearchOutput`: staged and promoted outputs.
- `ReviewDecision`: reviewer action history.
- `ResearchRunEvent`: append-only run telemetry.
- `ResearchToolCall`: budget/cost/tool-call accounting.
- `ReviewComment` and annotations: collaboration around outputs and claims.
- Collaboration records: collections, saved searches, shared project views,

notifications, audit views, and graph relationship records.

This matters because HAL's outputs are no longer isolated text blobs. Each run can be explained in terms of its program contract, source documents, retrieved memory, generated claims, tool calls, budget use, review decisions, and audit events.

Production Data Architecture

The implementation follows the production data architecture plan selected for v1:

- Local development defaults to SQLite and local filesystem object storage.
- Production database URLs are standardized on `postgresql+psycopg://...`.
- Alembic is the production schema-change path.
- S3-compatible storage is the production artifact store, with stable URIs:

`hal-local://...` locally and `s3://bucket/prefix/key` in production.

- Vectors are pgvector-first so embeddings, metadata, claims, chunks, outputs,

and review state can live in one operational database for v1.

- Embedding providers are abstracted, with deterministic fake embeddings in

tests and a deployment contract around embedding dimension.

This keeps local development lightweight while avoiding a local-only design.

Worker Execution Loop

The worker is now more than an output renderer. It implements the first real memory loop:

1. Pull a queued run from the shared store.
2. Validate budget, tool policy, and cancellation/timeout state.

3. Optionally execute live acquisition/search/download.
4. Process acquired or completed documents.
5. Chunk documents and create embeddings.
6. Extract first-pass claims.
7. Search memory across chunks, claims, and outputs.
8. Attach retrieval context to the run.
9. Generate source-rich staged outputs.
10. Record tool calls, acquisition telemetry, warnings, failures, and events.
11. Leave the run ready for human review.

The worker records per-paper acquisition outcomes such as searched, downloaded, processed, skipped, and failed. It also records LLM-call budget events and runtime budget checks, which gives operators a bounded execution model instead of an unbounded research script.

Output Framework

HAL now stages source-backed outputs through the store instead of writing opaque local artifacts. Current output targets include:

- Literature briefs.
- Evidence tables.
- Open questions.
- ADAM context JSON.
- Hypothesis cards.
- Experiment suggestions.
- Markdown and JSON export surfaces.
- Obsidian-compatible exports.
- Dashboard-friendly JSON.

The first-pass renderers have been upgraded toward source-rich renderers using real citations, extracted claims, and initial figure/table references. Output versioning and diffing are in place, so reviewers can compare revisions instead of treating regenerated text as a black box.

Review And Collaboration

Review is now a first-class workflow. Reviewers can:

- Inspect run telemetry and staged outputs.

- Promote a run.
- Reject a run.
- Request changes.
- Add comments and annotations on outputs and claims.
- View audit history for decisions and run events.

The lightweight browser review UI exposes review queue, output detail, comments, review actions, and audit dashboards. The Slack app service and HTTP gateway support slash commands and interactive button actions for review workflows. Sheets sync jobs produce non-CLI operational views for runs, review queue, outputs, and audit history.

This is the shape of the firm-wide research OS: one shared backend, multiple interfaces, and a consistent permission/audit model.

Slack And Sheets Surfaces

Slack is intended to be the fast collaboration surface:

- `/hal review <project_slug>` shows review-ready work.
- `/hal status <run_id>` reports run status.
- `/hal queue <project_slug> <objective>` is planned for run creation from

Slack.

- Buttons support promote, reject, request changes, and comments.
- Gateway routes verify Slack signatures and map Slack users to HAL users.

Google Sheets is intended to be the low-friction cockpit for non-CLI users:

- Run queue and status.
- Review queue.
- Output index.
- Audit dashboard.
- Cost/tool-call and acquisition telemetry views.

The current implementation supports native sync jobs for `runs`, `review_queue`, `outputs`, and `audit`. Sheets writeback for comments, decisions, and run queueing remains on the roadmap.

Deployment And Operations

The repository now includes deployment manifests for:

- API/gateway.

- Worker.
- Documentation.
- Postgres/pgvector.
- S3-compatible object storage.

It also includes backup/restore documentation for Postgres and object storage, environment profiles for local/staging/production, and CI coverage for the current rebuild surface.

The latest branch has a GitHub Actions quality gate that runs:

- Focused Ruff lint over rebuilt production/demo surfaces.
- Full pytest.
- Strict MkDocs build.
- Alembic migration smoke against a clean SQLite database.

The quality gate is passing on the rebuild branch.

What Tomorrow's Demo Should Prove

The demo does not need to pretend the product is finished. It should prove that the architecture is real and the loop is coherent:

1. HAL has shared memory, not one-off chat state.
2. Research runs are durable, bounded, and auditable.
3. Outputs are source-backed and staged for review.
4. Reviewers can act in the browser or Slack.
5. Sheets can expose the same state to non-CLI users.
6. Every meaningful action leaves an audit trail.
7. The branch has an automated quality gate.

That is enough to show the technical team that the rebuild has moved from plan to platform foundation.

Known Remaining Work

The remaining work is mostly about hardening, richer extraction, and production polish:

- Scheduled source refresh execution.
- Claim-level deduplication reports.
- Richer figure/table extraction and export rendering.
- Slack/Sheets production webhook polish.

- Sheets writeback for comments, decisions, and run queueing.
- Retention policy for PDFs, artifacts, logs, and generated outputs.
- Secrets management for providers, Slack, Sheets, and model APIs.
- Compliance review for copyrighted PDFs and generated summaries.
- Packaging/release process and changelog discipline.
- Broader type/lint cleanup beyond the rebuilt surfaces.

Demo Commands

Seed a repeatable demo project:

```
hal research demo-seed \
  --project-slug hal-demo \
  --owner bwisk@arc-pbc.com \
  --reviewer reviewer@example.com \
  --contributor researcher@example.com
```

Search shared memory:

```
hal research search-memory \
  "single crystal superalloy creep" \
  --project-slug hal-demo \
  --target chunks \
  --target claims \
  --target outputs \
  --json
```

Start browser review:

```
hal research review-ui --host 127.0.0.1 --port 9100
```

Start Slack gateway:

```
hal gateway http --host 127.0.0.1 --port 9101
```

Sync Sheets rows:

```
hal research sync-sheets hal-demo \
  --target review_queue \
  --spreadsheet-id <spreadsheet-id> \
  --range-name "Review Queue!A1" \
  --actor reviewer@example.com \
  --reviewer reviewer@example.com
```

Run the local quality gate:

```
python3 -m pytest -q
python3 -m mkdocs build --strict
tmpdir="$(mktemp -d)"
HAL9000_DATABASE__URL="sqlite:///${tmpdir}/hal9000-ci.db" python3 -m alembic upgrade head
rm -rf "${tmpdir}"
```

Bottom Line

HAL 9000 is now a credible foundation for a firm-wide research operating system: shared memory, bounded workers, source-backed outputs, review workflows, non-CLI collaboration surfaces, deployment scaffolding, and CI. The next phase is not a rewrite. It is hardening, richer extraction, production identity, retention/compliance, and deeper

integration with the firm's daily operating surfaces.